

Introducción a la computación

SESION 04

Teoría de Compiladores

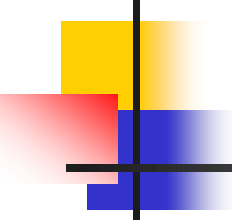
Introducción

UNIVERSIDAD NACIONAL DE INGENIERIA

Facultad de Ingeniería Industrial y de Sistemas

Ing. Miguel Angel Navarro Neyra

mnavarron@uni.edu.pe



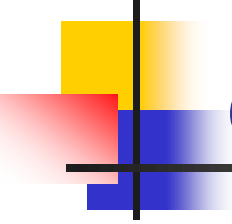
Agenda

■ Compilador

- Análisis léxico / Sintáctico / Semántico
- Análisis Sintáctico Descendente /Ascendente
- Tablas de tipos y símbolos
- Generación del código

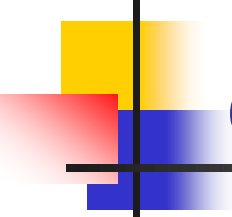
■ Intérprete

- Lenguajes de programación
- Tipos de lenguaje de programación



Compilador

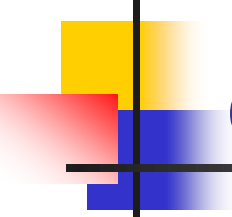
Historia



Compiladores

¿Qué es un compilador?

Un compilador es **un traductor** que transforma el **lenguaje fuente** (lenguaje de alto nivel) en **lenguaje objeto** (lenguaje máquina). La compilación también se puede dar de un lenguaje de alto nivel a un lenguaje de bajo nivel (ensamblador).



Compiladores

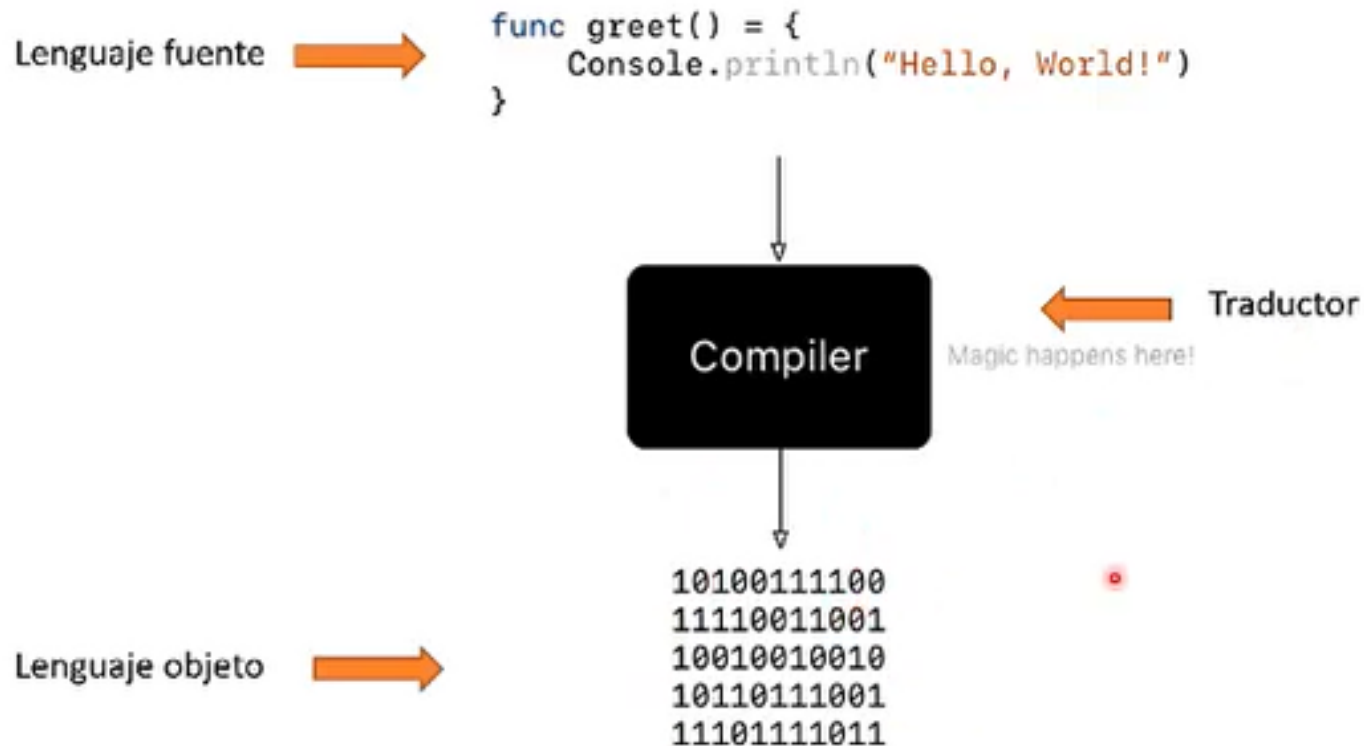
Lenguaje objeto

El **lenguaje objeto** es **directamente ejecutable** por la **computadora** u otro programa como una **máquina virtual**.

El compilador del **lenguaje C** transforma las instrucciones escritas en **C (fuente)** en lenguaje **máquina (objeto)**.

El compilador de **Java** convierte el código **en java (fuente)** en **Bytecodes (objeto)** que son ejecutados por la **Máquina Virtual de Java**.

Compiladores



Compiladores

Lenguaje fuente →

```
func greet() = {  
    Console.println("Hello, World!")  
}
```

Análisis Léxico

Análisis de Sintaxis

Análisis Semántico

Generación de Código
intermedio

Optimización del código

Generación del código
Final

Lenguaje objeto →

```
10100111100  
11110011001  
10010010010  
10110111001  
11101111011
```

Análisis Léxico

- En esta fase, el compilador escanea el código fuente y agrupa los caracteres en tokens significativos.
- Se identifican las unidades léxicas, como palabras reservadas, identificadores y operadores.
- Los comentarios se ignoran.

- Por ejemplo:

$x = y + 10$

Tokens:

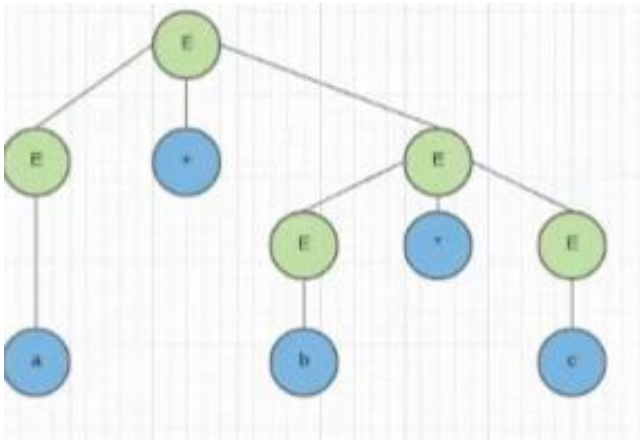
- x: Identificador
- =: Operador de asignación
- y: Identificador
- +: Operador de adición
- 10: Número

Análisis de Sintaxis

- Esta fase determina si el código sigue la estructura gramatical del lenguaje.
- Se construye un **árbol de análisis sintáctico** basado en las reglas del lenguaje.
- Verifica si las expresiones son sintácticamente correctas.

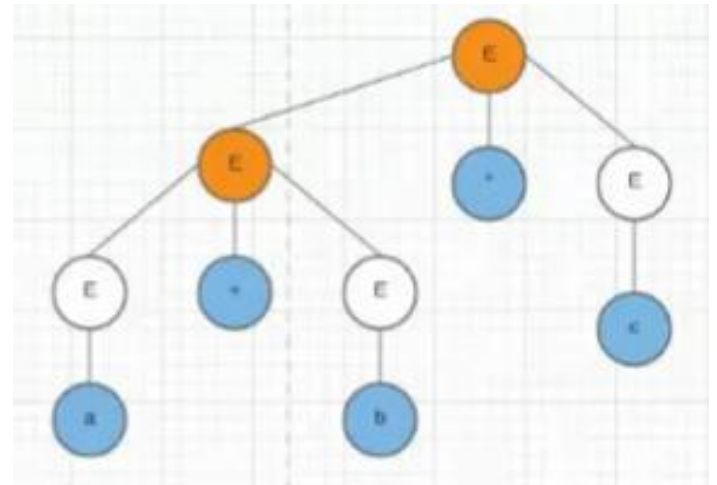
Análisis de Sintaxis

$a + b * c$



Ascendente

$(a + b) * c$



Descendente

Análisis Semántico

- Comprueba la coherencia semántica del código.
- Detecta errores como tipos incompatibles o variables no declaradas.
- Asegura que el programa tenga sentido desde el punto de vista del significado.

- Por ejemplo:

Si **X** es un identificador y,

y + 10 es una expresión,
entonces **x = y + 10** es
una declaración válida.

Generación de Código Intermedio

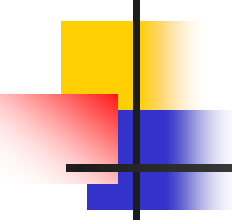
- Se crea una representación intermedia del programa.
- Puede ser un árbol de expresiones, código de tres direcciones, etc.

Generación de Código Intermedio

- Mejora el código intermedio para reducir tiempo de ejecución o uso de recursos.
- Realiza optimizaciones como eliminación de código muerto o simplificación de expresiones.

Generación de Código Final

- Se traduce el código intermedio a código de máquina específico para la arquitectura de la computadora.
- Se generan instrucciones ejecutables.



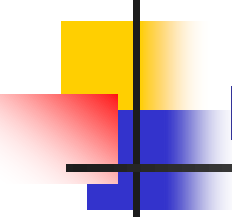
Agenda

- Compilador

- Análisis léxico / Sintáctico / Semántico
- Análisis Sintáctico Descendente /Ascendente
- Tablas de tipos y símbolos
- Generación del código

- Intérprete

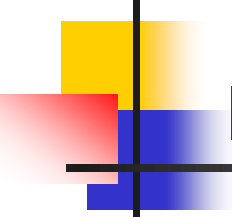
- Lenguajes de programación
- Tipos de lenguaje de programación



Interprete

¿Qué es un intérprete?

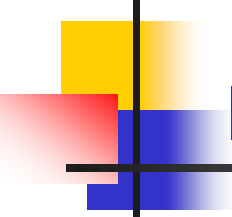
En ciencias de la computación, un **intérprete** es un programa **que ejecuta directamente instrucciones** escritas en un lenguaje de programación o scripting , sin requerir que **previamente hayan sido compiladas** en un programa de **lenguaje de máquina**.



Interprete

¿Qué es un intérprete?

El intérprete **transforma el programa de alto nivel** en un **lenguaje intermedio** que luego **ejecuta**, o podría analizar el código fuente de alto nivel y **luego ejecuta los comandos directamente**, lo hace **línea por línea** o declaración por declaración.



Interprete

¿Qué es un intérprete?

Python lleva una doble vida. Es un **intérprete** para scripts que puede ejecutar desde la línea de comandos o como aplicación si hace clic dos veces sobre su icono. Pero también es un **intérprete** interactivo que puede evaluar sentencias y expresiones arbitrarias.

Compilador vs Intérprete

En la mayoría de los casos, **un compilador es más favorable** ya que su **salida se ejecuta mucho más rápido** en comparación con una interpretación línea por línea. Sin embargo, dado que la interpretación ocurre por línea o declaración, **se puede detener en medio de la ejecución** para permitir **la modificación o depuración** del código.

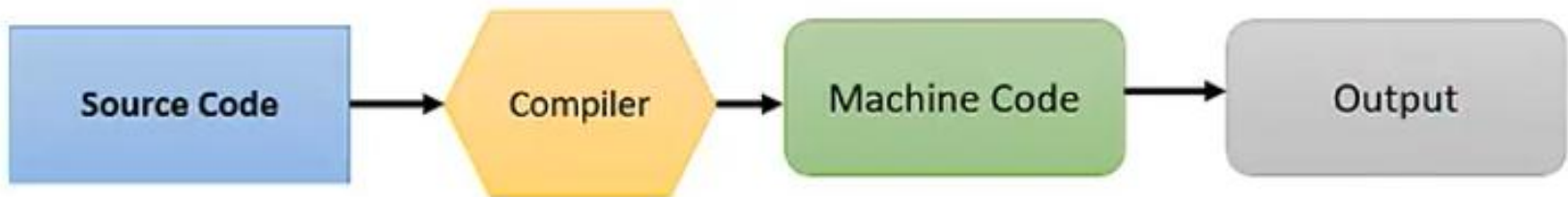
Compilador vs Intérprete

Ambos tienen **sus ventajas y desventajas** y no son mutuamente excluyentes; Esto significa que pueden usarse en conjunto, ya que la mayoría de los entornos de desarrollo integrados emplean tanto la compilación como la traducción para algunos lenguajes de alto nivel.

Interprete

Compilador vs Intérprete

How Compiler Works



How Interpreter Works



Interprete

Lenguajes compilados

**Código
Fuente**



Compilador

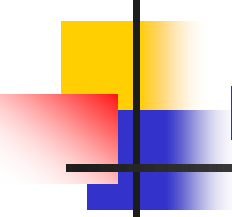


Ejecutable



Ejecutable





Interprete

Lenguajes Interpretados

Código Fuente

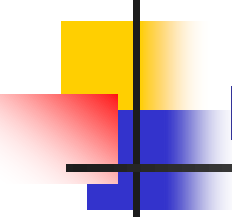


Código Fuente



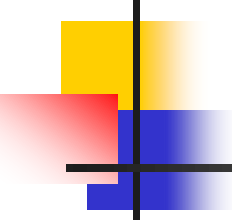
Interprete





Interprete

Compilados		Interpretados	
Ventajas	Desventajas	Ventajas	Desventajas
Preparados para ejecutarse	No son multiplataforma	Son multiplataforma	Se requiere un interprete
Usualmente más rápidos	Poco flexibles	Son más sencillos de probar	A menudo son más lentos
El código fuente es inaccesible	Se requiere un paso extra	Los errores se detectan más fácilmente	El código fuente es público

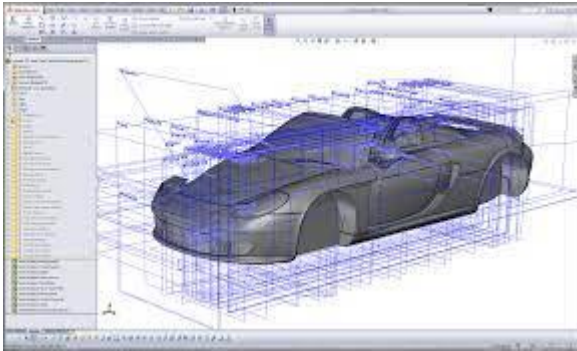


Agenda

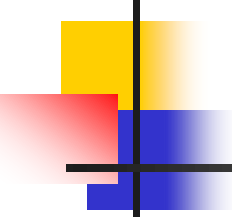
- Compilador
 - Análisis léxico / Sintáctico / Semántico
 - Análisis Sintáctico Descendente /Ascendente
 - Tablas de tipos y símbolos
 - Generación del código
- Intérprete
- Lenguajes de programación
- Tipos de lenguaje de programación

Lenguaje de programación

¿Qué es un programa?



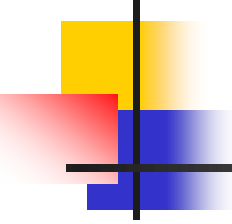
“Un programa de computadora es un conjunto de instrucciones que cumplen una finalidad concreta...”



Lenguaje de programación

¿Qué es un lenguaje de programación?

En informática, un **lenguaje de programación** es un **programa** destinado a la construcción de otros programas informáticos. Su nombre se debe a que comprende un **lenguaje formal** diseñado para organizar **algoritmos** y **procesos lógicos** que luego serán ejecutados por un ordenador o sistema informático. Estos lenguajes permiten controlar el comportamiento físico, lógico y la comunicación con el usuario humano.



Lenguaje de programación

¿Qué es un lenguaje de programación?

Los lenguajes de programación están compuestos por **símbolos** y **reglas sintácticas y semánticas**. A través de instrucciones y relaciones lógicas, se construye el **código fuente** de una **aplicación** o **software** específico. Además, estos lenguajes permiten el trabajo conjunto y coordinado de diversos programadores o arquitectos de software.

Lenguaje de programación

A# .NET
A# (Axiom)
A-0 System
A+
A++
ABAP
ABC
ABC ALGOL
ABLE
ABSET
ABSYS
ACC
Accent
Ace DASL
ACL2
ACT-III
Action!
B
Babbage
BAIL
Bash
BASIC
bc
BCPL
BeanShell
C
C--
C++ - ISO/IEC 14882
C# - ISO/IEC 23270
C/AL
Caché ObjectScript
C Shell
Caml
Candle
Cayenne
CDuce
Cecil
Cel
Cesil
Ceylon
CFML
Cg
Ch
Chapel
CHAIN
Charity

ActionScript
Ada
Adenine
Agda
Agilent VEE
Agora
AlIMMS
Alef
ALF
ALGOL 58
ALGOL 60
ALGOL 68
Alice
Alma-0
AmbientTalk
Amiga E
AMOS
Batch (Windows/Dos)
Bertrand
BETA
Bigwig
Bistro
BitC
BLISS
Blue
CHILL
CHIP-8
chomski
Chuck
CICS
Cilk
CL (IBM)
Claire
Clarion
Clean
Clipper
CLIST
Clojure
CLU
CMS-2
COBOL - ISO/IEC 1989
Cobra
CODE
CoffeeScript
Cola
ColdC

AMPL
APL
AppleScript
APRox
Argus
AspectJ
Assembly language
ATS
Ateji PX
AutoHotkey
Autocoder
AutoIt
AutoLISP / Visual LISP
Averest
AWK
Axum
Bon
Boo
Boomerang
Bourne shell (including bash and ksh)
BREW
BPE
Bullseye
COMAL
Common Programming Language (CPL)
Common Intermediate Language (CIL)
Common Lisp (also known as CL)
COMPASS
Component Pascal
COMIT
Constraint Handling Rules (CHR)
Converge
Coral 66
Corn
CorVision
Coq
COWSEL
CPL
csh
CSP
Csound
Curl
Curry
Curlene

Python

C

Java

C++

C#

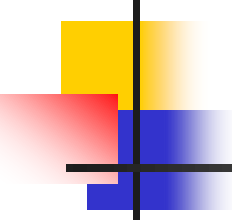
Visual Basic

JavaScript

Perl

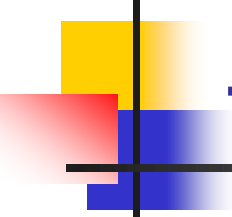
Ruby

PHP



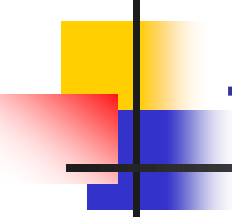
Agenda

- Compilador
 - Análisis léxico / Sintáctico / Semántico
 - Análisis Sintáctico Descendente /Ascendente
 - Tablas de tipos y símbolos
 - Generación del código
- Intérprete
- Lenguajes de programación
- Tipos de lenguaje de programación

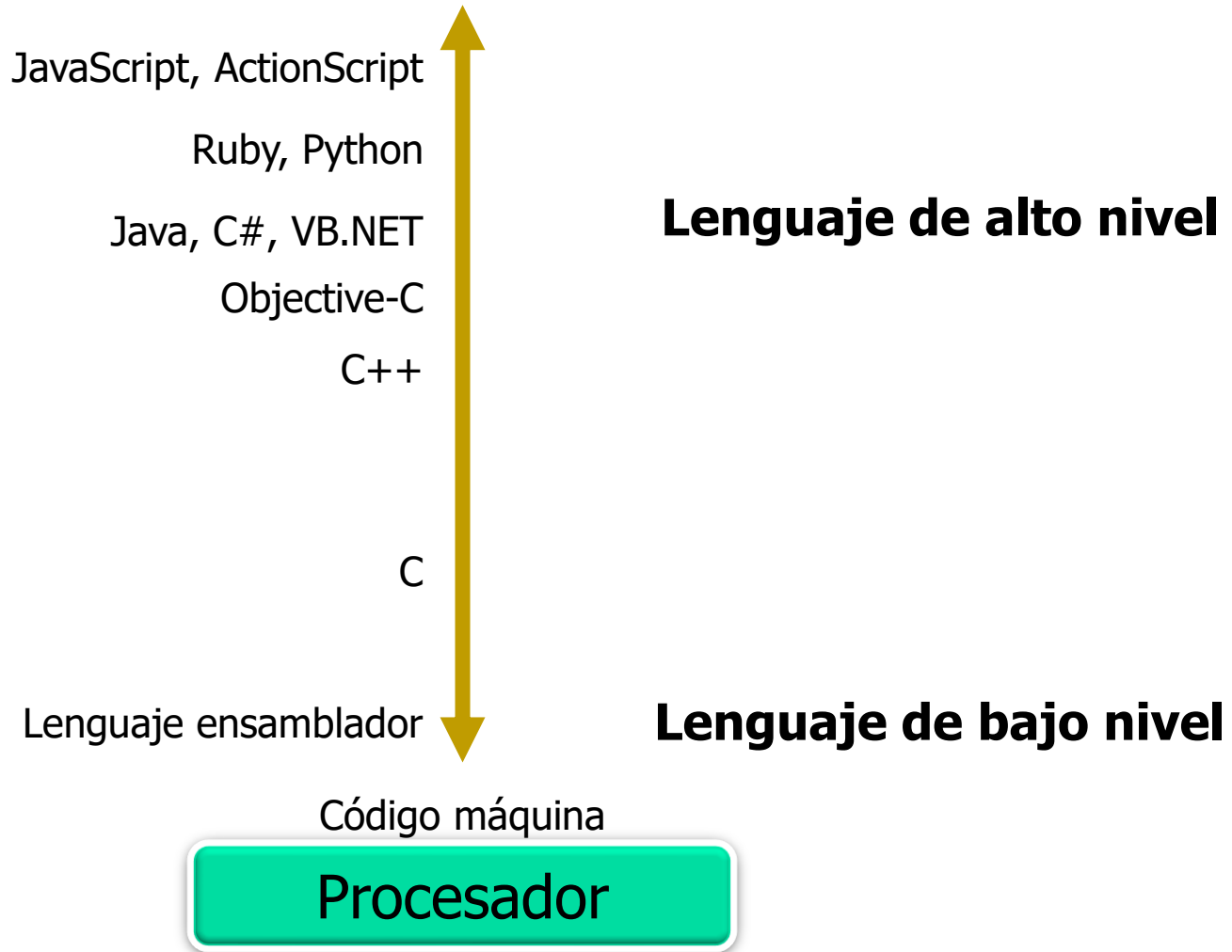


Tipos de lenguajes de programación

- **Lenguajes de bajo nivel:** Diseñados para un hardware específico, no son portables a otros computadores.
- **Lenguajes de alto nivel:** Más universales, pueden emplearse en diversos tipos de sistemas.
- **Lenguajes de nivel medio:** Se ubican entre los dos anteriores, permitiendo operaciones de alto nivel y gestión local de la arquitectura del sistema.

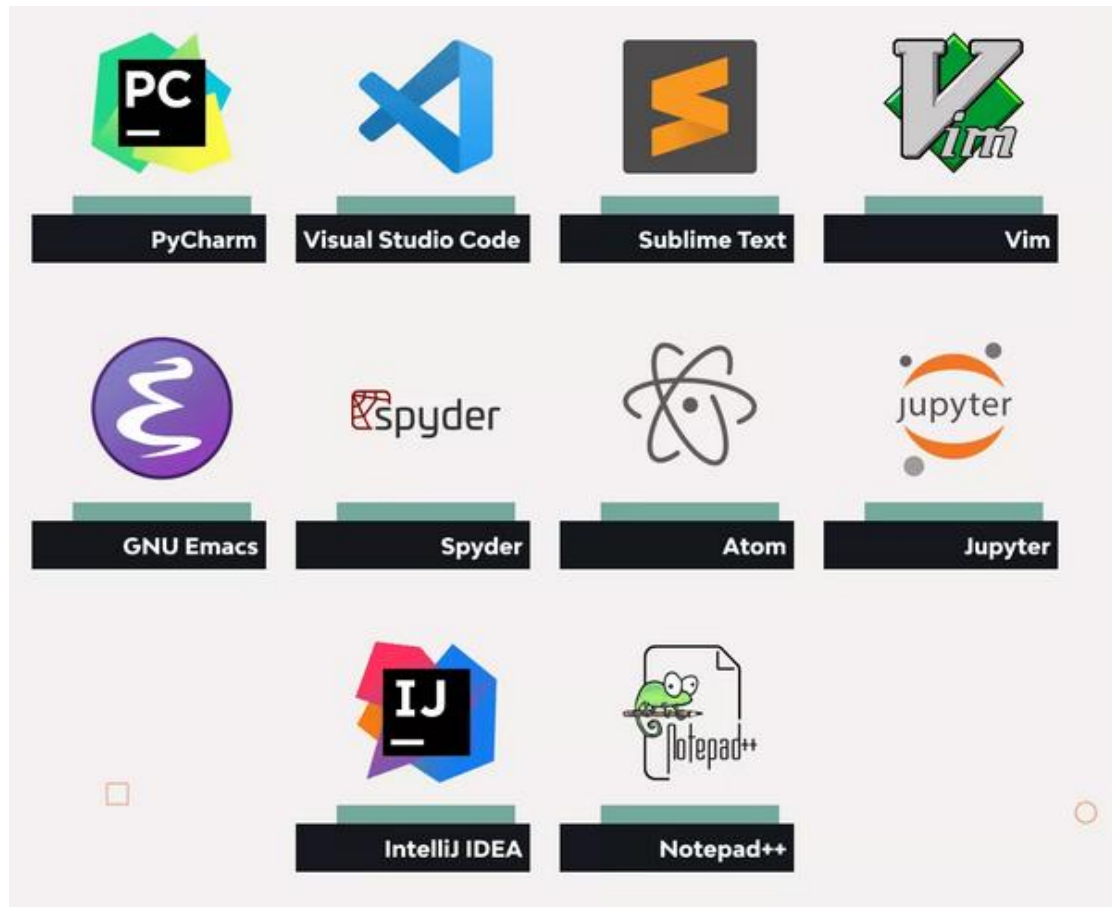


Tipos de lenguajes de programación

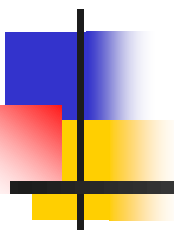


Tipos de lenguajes de programación

Entornos de desarrollo integrados (IDE)



Introducción a la computación



FIN SESION 04